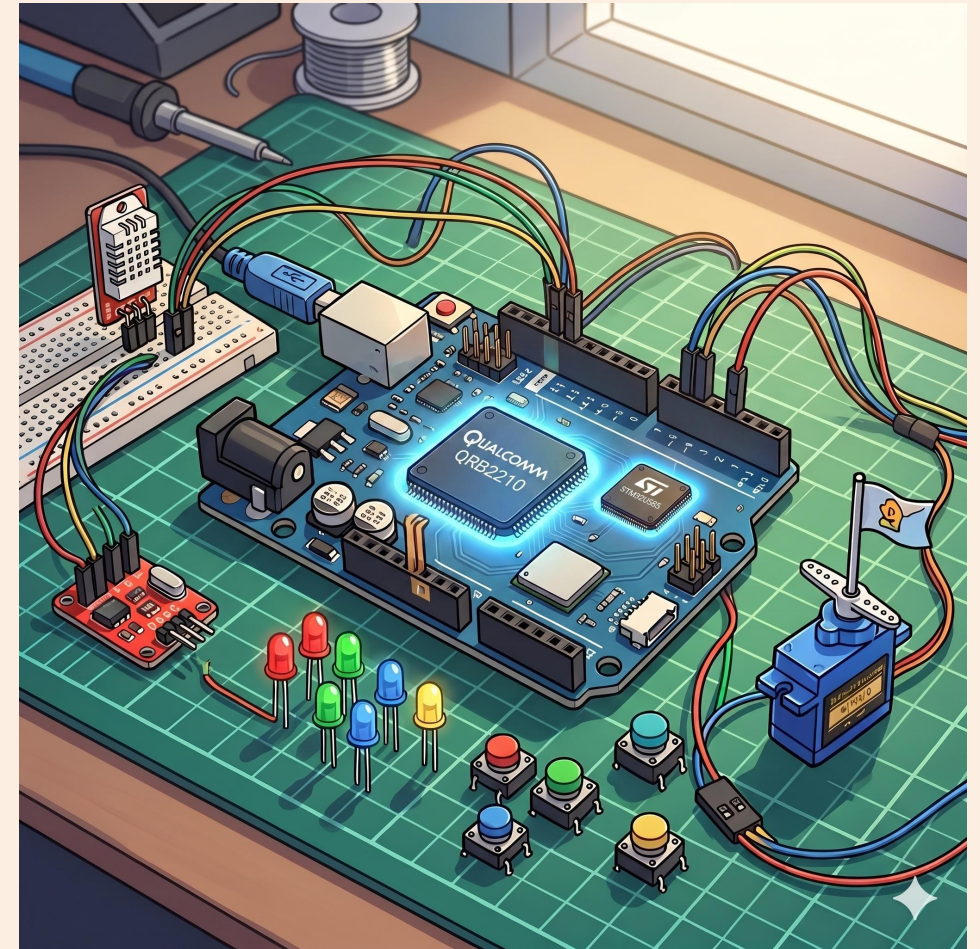


Generative AI at the Edge

Running small language models with llama.cpp

No internet · no API keys · no cloud · nothing leaves the board

Marcelo Rovai · UNIFEI



WHERE WE ARE GOING

A language model answering on 3.1 watts, with the network unplugged.

About 6 to 7 tokens per second, ~54 °C, no heatsink and no fan.



```
marcelo_rovai — arduino@uno-q: ~/llama.cpp — ssh arduino@192.168.5.114 — 84x45
arduino@uno-q:~/llama.cpp$ cd ~/llama.cpp

./build/bin/llama-cli \
  --model ~/models/Qwen_Qwen3.5-0.8B-Q8_0.gguf \
  --threads 4 \
  --ctx-size 1024 \
  --temperature 0.7 \
  --top-p 0.8 \
  --top-k 20 \
  --min-p 0.0 \
  --presence-penalty 1.5 \
  --repeat-penalty 1.0 \
  --reasoning off \
  --reasoning-budget 0

Loading model...

llama.cpp

build      : b9330-328874d05
model      : Qwen_Qwen3.5-0.8B-Q8_0.gguf
modalities : text

available commands:
/exit or Ctrl+C  stop or exit
/regen           regenerate the last response
/clear          clear the chat history
/read <file>     add a text file
/glob <pattern>  add text files using globbing pattern

[> What is the capital of Brazil. Answer with one word.]

Brasilia

[ Prompt: 10.2 t/s | Generation: 7.8 t/s ]

> █
```

Six stages, one local model at the end

1

Constraints

Memory, storage, model choice and quantization — decided before you download.

2

Build

Swap, build tools, a shallow clone, CMake, and 20 minutes of four cores at 100%.

3

llama-cli

One prompt, one answer — and the one flag that decides if this board is usable.

4

Server

Load the model once, expose an OpenAI-compatible HTTP API, query it with curl.

5

Python

The openai client library, then raw HTTP, so you see what the library does.

6

Limits

Measured throughput, the four failure modes, and when 0.8B stops being enough.

Three ways to talk to a local model

STAGE 3

llama-cli

One-shot prompts. No server, no extra process.

The fastest way to sanity-check a model before building anything on top of it.

STAGE 4

llama-server

An OpenAI-compatible HTTP API.

Loads the GGUF into RAM once and serves any client that can speak JSON.

STAGE 5

Python

The standard openai client library, then raw HTTP — so you can see what the library is doing underneath.

Prerequisite: Chapter 1. You need SSH access to the board. Nothing else is assumed.

STAGE 1 OF 6

What is realistic on this board

Memory, storage, model choice and quantization — decided before you download anything.

1

FRAMING

Why small language models change the picture

WHAT WE TAUGHT BEFORE

Classic TinyML

One model, one task. A wake-word detector. An image classifier.

Milliseconds of latency, kilobytes of RAM — and a retraining cycle for every new task.

WHAT CHANGES HERE

A small language model

One sub-1B model can classify, summarise, reformat, translate and reason about structured input.

Seconds of latency, and it needs Linux and a couple of GB of RAM.

'Small' means roughly 100 M to 7 B parameters — pruned, distilled and quantized to run CPU-only.

THE TRADE

What you pay, and what you get

seconds

not milliseconds

the latency you are buying with

≥ 2 GB RAM

+ Linux

the platform floor

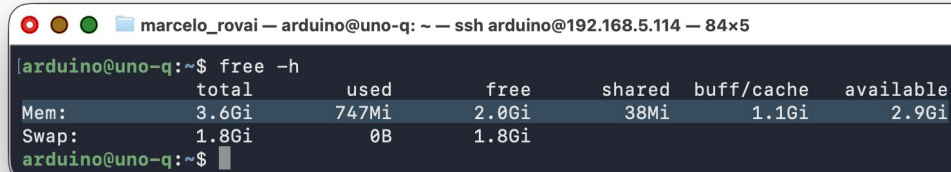
one model

many tasks

what you get in return

CONSTRAINT 1

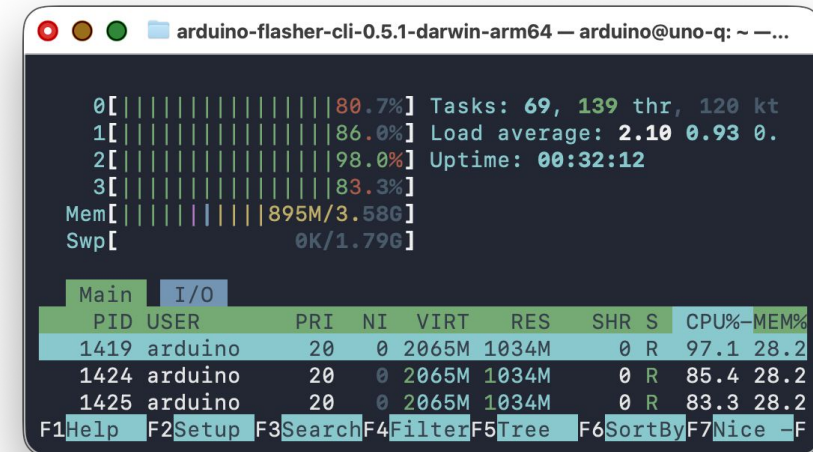
The memory reality on a 4 GB board



```

arduino@uno-q:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.6Gi         747Mi         2.0Gi          38Mi          1.1Gi         2.9Gi
Swap:          1.8Gi           0B          1.8Gi
  
```

free -h after boot, nothing running



```

0[|||||80.7%] Tasks: 69, 139 thr, 120 kt
1[|||||86.0%] Load average: 2.10 0.93 0.
2[|||||98.0%] Uptime: 00:32:12
3[|||||83.3%]
Mem[|||||895M/3.58G]
Swp[|||||0K/1.79G]

Main I/O
PID USER PRI NI VIRT RES SHR S CPU% MEM%
1419 arduino 20 0 2065M 1034M 0 R 97.1 28.2
1424 arduino 20 0 2065M 1034M 0 R 85.4 28.2
1425 arduino 20 0 2065M 1034M 0 R 83.3 28.2
F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice F8
  
```

htop while a 0.8B Q8 model generates

Model + context $\approx$ 1.1 GB, leaving $\sim$ 1.7 GB. The 2 GB board needs a smaller model — SmolLM2-135M or 360M.

CONSTRAINT 2

Check your partition layout first

```
$ df -h /home/arduino
```

```
# do this BEFORE you download anything
```

Two layouts are in the wild. Which one you have decides whether you need the shallow clone and the build-tree cleanup at all.

CONSTRAINT 2

The two layouts you might have

OLDER IMAGES

One 9.8 GB partition

/ and /home share it, with about 830 MB free after boot.

Disk management matters: use the shallow clone, and delete the build tree afterwards.

NEWER IMAGES

/home on its own 18 GB

Source, models and projects all land there; the 9.8 GB root stays untouched.

You can skip the cleanup steps entirely.

You need at least 5 GB free to complete the build and download a model.

MODEL CHOICE

Why Qwen3.5-0.8B

Qwen3.5 Family Comparison on Shared Benchmarks

	397B-A17B	122B-A10B	35B-A3B	27B	9B	4B	2B	0.8B
Overall	100%	97%	95%	97%	90%	85%	72%	54%
Text Overall	100%	97%	94%	97%	90%	84%	64%	41%
General Agent	100%	95%	93%	93%	91%	81%	58%	24%
Long Context	100%	96%	89%	96%	90%	81%	49%	24%
Instruction Following	100%	97%	93%	98%	88%	82%	63%	37%
Multilingualism	100%	98%	95%	98%	90%	84%	64%	46%
Knowledge	100%	98%	96%	97%	92%	88%	73%	49%
Visual Overall	100%	97%	95%	97%	90%	86%	76%	62%
STEM and Puzzle	100%	93%	89%	94%	79%	76%	61%	43%
Medical VQA	100%	99%	94%	95%	87%	81%	72%	62%
Video Understanding	100%	99%	97%	98%	94%	90%	78%	62%
Visual Agent	100%	107%	105%	107%	99%	92%	83%	71%
Spatial Intelligence	100%	97%	96%	97%	91%	87%	77%	67%
General VQA	100%	97%	96%	95%	92%	88%	81%	68%
Text Recognition and Document Understanding	100%	98%	98%	98%	95%	92%	85%	71%

≤ 80% ≥ 103%
Color scale: 80%–103%. Values below 80% or above 103% are clipped.

The Qwen3.5 Small family

Released February 2026 · Apache 2.0 · 201 languages · designed for edge inference.

OPTIONS

Candidate models for this board

Model	Params	Size	Notes
SmolLM2-135M-Instruct Q4_K_M	135 M	~95 MB	Very fast, limited reasoning. Routing and classification only.
SmolLM2-360M-Instruct Q4_K_M	360 M	~230 MB	Balanced. The fallback if 0.8B is too tight.
Qwen3.5-0.8B Q8_0	800 M	~880 MB	The primary choice. Best quality-per-parameter.
Qwen3.5-2B (Q4-Q8)	2 B	1.25–2 GB	Excellent, noticeably slower. Quality over snappiness.

Have the fallback ready: a student on a 2 GB board or short of disk stays in the lesson with SmolLM2-360M instead of stuck on a download.

GETTING THE WEIGHTS

Download a GGUF

```
$ mkdir -p ~/models/Qwen3.5-0.8B-GGUF
$ cd ~/models/Qwen3.5-0.8B-GGUF

$ wget -c "huggingface.co/unsloth/Qwen3.5-0.8B-GGUF/\
    resolve/main/Qwen3.5-0.8B-Q8_0.gguf"
```

One directory per model. You will have several quantizations of the same weights within an hour, and telling them apart by filename stops working quickly.

CONSTRAINT 3

At sub-1B scale, Q4 is aggressive.

The quantization error compounds in a smaller model — there is less redundancy to absorb it. At 7B, Q4 is nearly free. At 0.8B it removes information the model cannot afford to lose.

CONSTRAINT 3

What to do about it

1

Start with Q4_K_M

~550 MB. Use it just to verify the whole workflow fits on your board at all.

2

Then try Unsloth Dynamic

UD-Q4_K_XL upcasts critical layers to 8 or 16 bits, near Q4 file size.

3

Settle on Q8_0 if you can

~850 MB. Meaningfully better quality — and what this chapter uses.

4

Prefer min_p over top_p

min_p 0.05 with temp 0.7 adapts to the actual distribution; top_p at low temperature makes repetition almost inevitable at this size.

If you must ship Q4: strong few-shot prompting, response_format json_object, presence_penalty 1.5, low max_tokens.

STAGE 2 OF 6

Building llama.cpp from source

Swap, build tools, a shallow clone, CMake — and 20 minutes of four A53 cores at 100%.

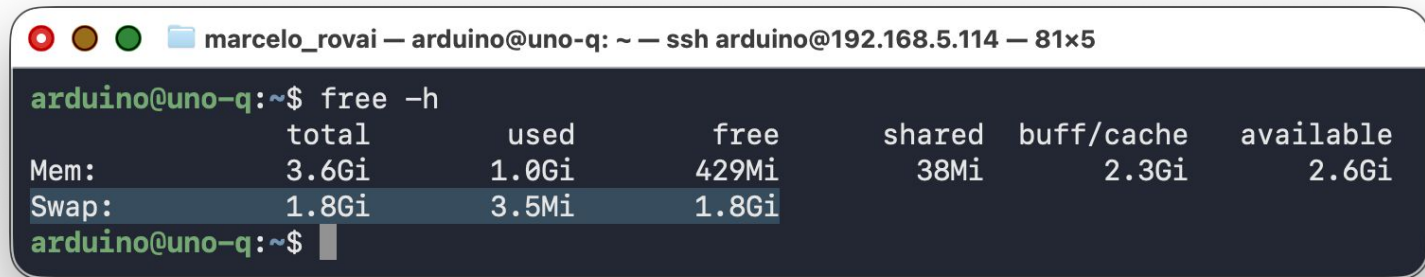
2

PREPARATION

Check the ground first

```
$ uname -m      # aarch64
$ free -h       # ~3.6 Gi total, Swap: ~1.8 Gi
$ df -h /       # root partition space
```

If Swap shows 0 B, add it on the next slide before you build. The OOM killer during model load is what you are protecting against.



```
marcelo_rovai — arduino@uno-q: ~ — ssh arduino@192.168.5.114 — 81x5
arduino@uno-q:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:          3.6Gi         1.0Gi         429Mi        38Mi         2.3Gi         2.6Gi
Swap:          1.8Gi         3.5Mi         1.8Gi
arduino@uno-q:~$
```

free -h — swap is present

PREPARATION

Add 2 GB of swap, if it is missing

```
$ sudo fallocate -l 2G /home/swapfile
$ sudo chmod 600 /home/swapfile
$ sudo mkswap /home/swapfile
$ sudo swapon /home/swapfile

# make it survive a reboot
$ echo '/home/swapfile none swap sw 0 0' | \
    sudo tee -a /etc/fstab
```

Only needed on older images. Newer ones ship with swap already configured.

PREPARATION

Install the build tools

```
$ sudo apt update  
$ sudo apt install -y build-essential cmake git \  
    pkg-config libcurl4-openssl-dev
```

libcurl4-openssl-dev is what lets llama-cli and llama-server download a GGUF themselves.

THE BUILD

Shallow clone, then CMake

```
$ cd /home/arduino
$ git clone --depth 1 https://github.com/ggml-org/llama.cpp
$ cd llama.cpp

$ cmake -B build \
    -DLLAMA_CURL=ON \
    -DGGML_NATIVE=ON \
    -DCMAKE_BUILD_TYPE=Release
$ cmake --build build --config Release -j4
```

20–25 minutes with -j4. If the board freezes or the build dies, drop to -j2 — slower, but it stays under the OOM threshold.

THE BUILD

What those four flags are doing

1 **--depth 1**

A full clone pulls ~400 MB of git history you will never read. Shallow is 80–100 MB.

2 **-DLLAMA_CURL=ON**

Lets llama-cli and llama-server download models directly.

3 **-DGGML_NATIVE=ON**

Detects the A53's NEON SIMD capabilities and emits the right CPU intrinsics.

4 **-j4**

All four cores. Drop to -j2 if the build runs out of memory.

Verify when it finishes: `ls build/bin/llama-cli build/bin/llama-server`

VERIFY

You should have two binaries



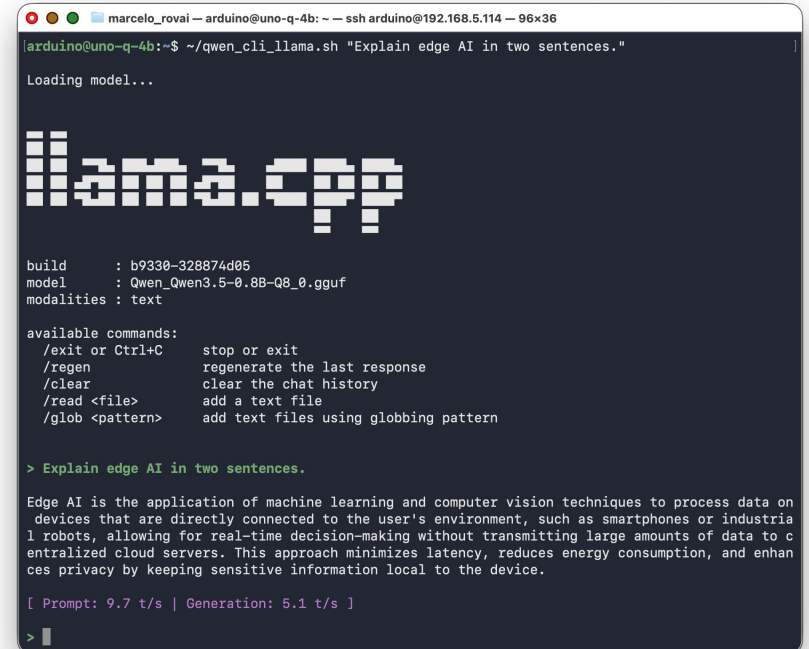
```
marcelo_rovai — arduino@uno-q: ~/llama.cpp — ssh arduino@192.168.5.114 — 84x6  
[arduino@uno-q:~/llama.cpp$ ls build/bin/llama-cli build/bin/llama-server  
build/bin/llama-cli build/bin/llama-server  
[arduino@uno-q:~/llama.cpp$ ./build/bin/llama-cli --version  
version: 9330 (328874d05)  
built with GNU 14.2.0 for Linux aarch64  
arduino@uno-q:~/llama.cpp$
```

llama-cli --version — the build worked

SHORTCUT

Or skip the build entirely.

llama.cpp publishes pre-built aarch64 Linux binaries on its GitHub Releases page. Download the tarball, extract llama-server and llama-cli, and you're done.



```
marcelo_rovai — arduino@uno-q-4b: ~ — ssh arduino@192.168.5.114 — 96x36
arduino@uno-q-4b:~$ ~/qwen_cli_llama.sh "Explain edge AI in two sentences."
Loading model...

llama.cpp

build      : b9330-328874d05
model      : Qwen_Qwen3.5-0.8B-Q8_0.gguf
modalities : text

available commands:
/exit or Ctrl+C  stop or exit
/regen           regenerate the last response
/clear          clear the chat history
/read <file>     add a text file
/glob <pattern>  add text files using globbing pattern

> Explain edge AI in two sentences.

Edge AI is the application of machine learning and computer vision techniques to process data on devices that are directly connected to the user's environment, such as smartphones or industrial robots, allowing for real-time decision-making without transmitting large amounts of data to centralized cloud servers. This approach minimizes latency, reduces energy consumption, and enhances privacy by keeping sensitive information local to the device.

[ Prompt: 9.7 t/s | Generation: 5.1 t/s ]
> █
```

HOUSEKEEPING

Reclaim ~690 MB after the build

```
$ cd /home/arduino && mkdir -p llama-runtime
$ cp llama.cpp/build/bin/llama-server llama-runtime/
$ cp llama.cpp/build/bin/llama-cli      llama-runtime/
$ cp llama.cpp/build/bin/*.so*          llama-runtime/

# verify it stands alone BEFORE deleting anything
$ cd llama-runtime && LD_LIBRARY_PATH=. ./llama-cli --version

$ rm -rf /home/arduino/llama.cpp      # saves ~690 MB
```

Only on the older single-partition image. The LD_LIBRARY_PATH test is not optional — find out which shared objects are copied while the source tree is still present.

STAGE 3 OF 6

llama-cli: the sanity check

3

One prompt, one answer, no server — and one flag that decides whether the board is usable.

THE CRITICAL FLAG

Turn thinking mode off. Always.

Qwen3.5 is a hybrid reasoning model and defaults to thinking ON. On this hardware that means 30–60 seconds of internal chain-of-thought before any output — and frequent loops that never reach a conclusion.

THE CRITICAL FLAG

The two flags that turn it off

```
--reasoning off --reasoning-budget 0
```

The older `--chat-template-kwarg {"enable_thinking":false}` is deprecated and emits a warning. Replace it wherever you find it.

SAMPLING

Recommended parameters for non-thinking mode

Parameter	Value	Purpose
temperature	0.7 – 1.0	Moderate creativity for general tasks
top_p	0.8	Nucleus sampling
top_k	20	Limit the token pool
min_p	0.0	Disabled by default — but see the note on small models
presence_penalty	1.5	Prevent repetition loops — critical at this size
repetition_penalty	1.0	Disabled — presence_penalty handles it

presence_penalty = 1.5 is the single most load-bearing value in this chapter. Set it everywhere.

FIRST INFERENCE

Run a prompt

```
$ ./build/bin/llama-cli \  
  --model ~/models/Qwen3.5-0.8B-Q8_0.gguf \  
  --threads 4 --ctx-size 1024 \  
  --temperature 0.7 --top-p 0.8 --top-k 20 \  
  --presence-penalty 1.5 --repeat-penalty 1.0 \  
  --reasoning off --reasoning-budget 0 \  
  -p "What is the capital of Brazil? One word."
```

Expect 5–10 tokens/s. That is the real number on this board — usable, not fast.



```
marcelo_rovai ~ arduino@uno-q: ~/llama.cpp -- ssh arduino@192.168.5.114 -- 84x45  
arduino@uno-q:~/llama.cpp$ cd ~/llama.cpp  
./build/bin/llama-cli \  
  --model ~/models/Qwen3.5-0.8B-Q8_0.gguf \  
  --threads 4 \  
  --ctx-size 1024 \  
  --temperature 0.7 \  
  --top-p 0.8 \  
  --top-k 20 \  
  --min-p 0.0 \  
  --presence-penalty 1.5 \  
  --repeat-penalty 1.0 \  
  --reasoning off \  
  --reasoning-budget 0  
Loading model...  
  
llama.cpp  
  
build      : b9330-328874d05  
model      : Qwen3.5-0.8B-Q8_0.gguf  
modalities : text  
  
available commands:  
/exit or Ctrl+C  stop or exit  
/regen           regenerate the last response  
/clear           clear the chat history  
/read <file>     add a text file  
/glob <pattern>  add text files using globbing pattern  
  
> What is the capital of Brazil. Answer with one word.  
  
Brasilia  
  
[ Prompt: 10.2 t/s | Generation: 7.8 t/s ]  
  
> █
```

First local inference

THERMALS

No heatsink. No fan. ~54 °C.



All four cores at 100%, temperature during inference

~30 °C idle → ~54 °C inference → ~62 °C on sustained long answers. Throttle threshold is 70–80 °C.

ERGONOMICS

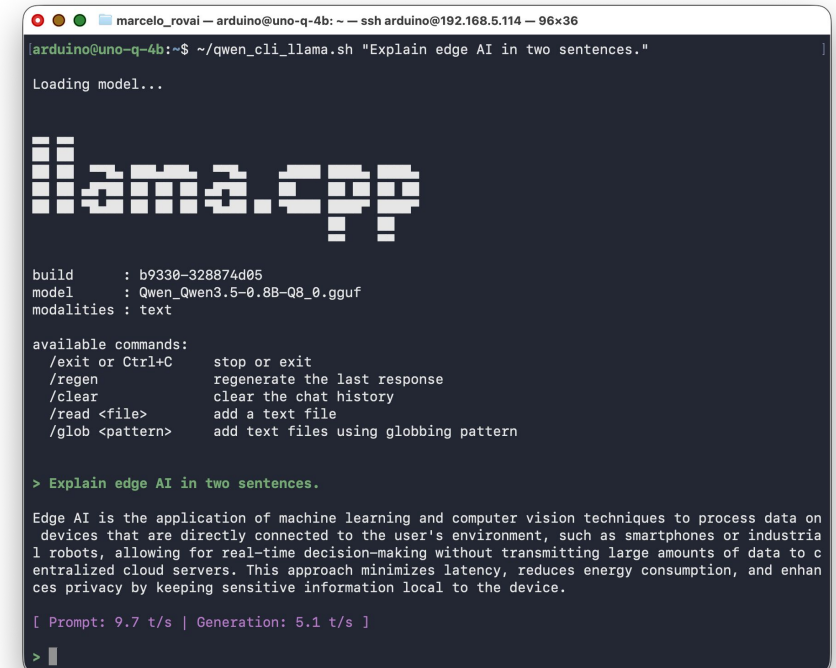
Wrap it in a script

```
#!/usr/bin/env bash
# ~/qwen_cli_llama.sh

MODEL=~/.models/Qwen3.5-0.8B-Q8_0.gguf
LLAMA=~/.llama.cpp/build/bin/llama-cli

"$LLAMA" --model "$MODEL" --threads 4 --ctx-size 1024 \
--temperature 0.7 --top-p 0.8 --top-k 20 \
--presence-penalty 1.5 --repeat-penalty 1.0 \
--reasoning off --reasoning-budget 0 -p "$@"
```

chmod +x it, add ~/ to PATH in .bashrc, and it becomes:
qwen_cli_llama.sh "your prompt"



```
marcelo_rovai — arduino@uno-q-4b: ~ — ssh arduino@192.168.5.114 — 96x36
[arduino@uno-q-4b:~$ ~/qwen_cli_llama.sh "Explain edge AI in two sentences."

Loading model...

llama.cpp

build      : b9330-328874d05
model      : Qwen_Qwen3.5-0.8B-Q8_0.gguf
modalities : text

available commands:
/exit or Ctrl+C  stop or exit
/regen           regenerate the last response
/clear           clear the chat history
/read <file>     add a text file
/glob <pattern>  add text files using globbing pattern

> Explain edge AI in two sentences.

Edge AI is the application of machine learning and computer vision techniques to process data on
devices that are directly connected to the user's environment, such as smartphones or industria
l robots, allowing for real-time decision-making without transmitting large amounts of data to c
entralized cloud servers. This approach minimizes latency, reduces energy consumption, and enhan
ces privacy by keeping sensitive information local to the device.

[ Prompt: 9.7 t/s | Generation: 5.1 t/s ]

> 
```

The script in use

ONE CAVEAT

For anything you build on, use llama-server.

llama-cli is less reliable than the server at suppressing Qwen3.5 reasoning traces — a known issue. Use the CLI for quick experiments; target the server for real work.

STAGE 4 OF 6

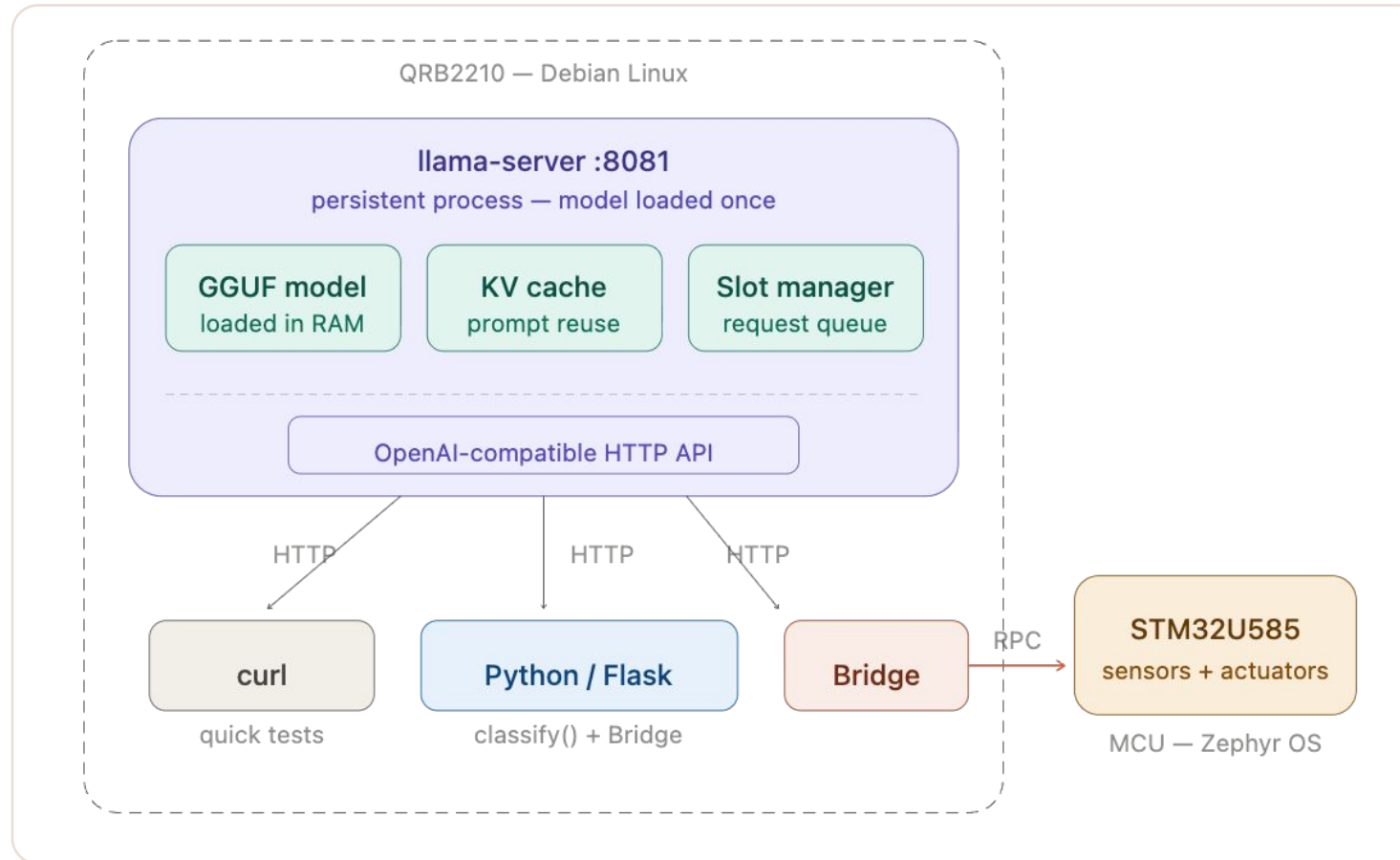
llama-server: the HTTP API

Load the GGUF into RAM once, expose an OpenAI-compatible endpoint, query it from anything.

4

THE SHAPE OF IT

One model in RAM, many clients



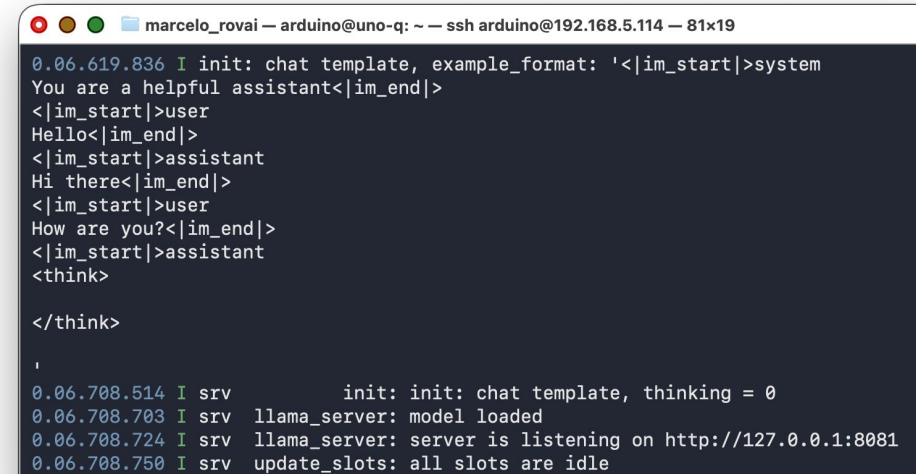
llama-server: load once, serve anything that speaks HTTP

TERMINAL 1

Start the server in the foreground

```
$ ./build/bin/llama-server \  
  --model ~/models/Qwen3.5-0.8B-Q8_0.gguf \  
  --threads 4 --ctx-size 1024 \  
  --port 8081 \  
  --reasoning off --reasoning-budget 0 \  
  --alias qwen3.5-0.8b
```

Do not close this terminal. The server lives as long as this process. In the log look for: the model loading · thinking = 0 · HTTP server listening.



```
0.06.619.836 I init: chat template, example_format: '<|im_start|>system  
You are a helpful assistant<|im_end|>  
<|im_start|>user  
Hello<|im_end|>  
<|im_start|>assistant  
Hi there<|im_end|>  
<|im_start|>user  
How are you?<|im_end|>  
<|im_start|>assistant  
<think>  
  
</think>  
,  
0.06.708.514 I srv          init: init: chat template, thinking = 0  
0.06.708.703 I srv          llama_server: model loaded  
0.06.708.724 I srv          llama_server: server is listening on http://127.0.0.1:8081  
0.06.708.750 I srv          update_slots: all slots are idle
```

Terminal 1 — the server log

THE FLAG TO INTERNALISE

--alias gives the model a stable name.

curl, Python and the WebUI can always ask for "model": "qwen3.5-0.8b", no matter which GGUF file is loaded behind it. That matters the moment you start swapping models — as Chapters 3 and 5 do.

TERMINAL 2

Is it alive?

```
$ curl -s http://127.0.0.1:8081/health | python3 -m json.tool
{"status": "ok"}
```

```
marcelo_rovai — arduino@uno-q: ~/llama.cpp — ssh arduino@192.168.5.114 — 86x25
er_second":5.829810346553076}}arduarduino@uno-q:~/llama.cpp$ curl http://127.0.0.1:808
1/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3.5-0.8b",
  "messages": [
    {"role": "system", "content": "You are concise. Answer in one sentence."},
    {"role": "user", "content": "What is TinyML?"}
  ],
  "max_tokens": 80,
  "temperature": 0.7,
  "top_p": 0.8,
  "presence_penalty": 1.5
}'
{"choices":[{"finish_reason":"stop","index":0,"message":{"role":"assistant","content":
"TinyML is a computer vision and natural language processing framework designed to dep
loy lightweight neural networks on resource-constrained devices like mobile phones, en
abling low-cost AI applications without requiring expensive cloud infrastructure."}}],
"created":1779827553,"model":"Qwen_Qwen3.5-0.8B-Q8_0.gguf","system_fingerprint":"b9330
-328874d05","object":"chat.completion","usage":{"completion_tokens":38,"prompt_tokens"
:31,"total_tokens":69,"prompt_tokens_details":{"cached_tokens":0},"id":"chatcmpl-zLcQ
IfLZWjZaRhtJpuUP44zFrbAbZB6z","timings":{"cache_n":0,"prompt_n":31,"prompt_ms":2974.55
,"prompt_per_token_ms":95.95322580645161,"prompt_per_second":10.421744465549411,"predi
cted_n":38,"predicted_ms":6738.869,"predicted_per_token_ms":177.33865789473683,"predic
arduarduino@uno-q:~/llama.cpp$
```

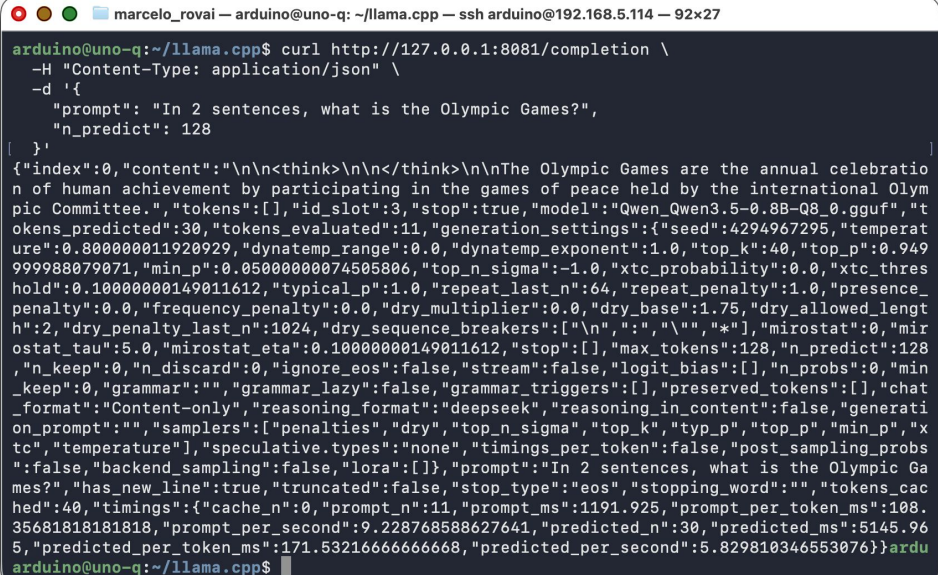
/health

TERMINAL 2

The raw completion endpoint

```
$ curl http://127.0.0.1:8081/completion \
  -H "Content-Type: application/json" \
  -d '{"prompt": "What is TinyML?",
      "n_predict": 128}'
```

This is llama.cpp's own endpoint — useful, but not the one your code should target.



```
marcelo_rovai — arduino@uno-q: ~/llama.cpp — ssh arduino@192.168.5.114 — 92x27
arduino@uno-q:~/llama.cpp$ curl http://127.0.0.1:8081/completion \
-H "Content-Type: application/json" \
-d '{
  "prompt": "In 2 sentences, what is the Olympic Games?",
  "n_predict": 128
}'
{"index":0,"content":"\n\n<think>\n\n</think>\n\nThe Olympic Games are the annual celebration of human achievement by participating in the games of peace held by the international Olympic Committee.","tokens":[],"id_slot":3,"stop":true,"model":"Qwen_Qwen3.5-0.8B-Q8_0.gguf","tokens_predicted":30,"tokens_evaluated":11,"generation_settings":{"seed":4294967295,"temperature":0.800000011920929,"dynatemp_range":0.0,"dynatemp_exponent":1.0,"top_k":40,"top_p":0.94999988079071,"min_p":0.05000000074505806,"top_n_sigma":-1.0,"xtc_probability":0.0,"xtc_threshold":0.10000000149011612,"typical_p":1.0,"repeat_last_n":64,"repeat_penalty":1.0,"presence_penalty":0.0,"frequency_penalty":0.0,"dry_multiplier":0.0,"dry_base":1.75,"dry_allowed_length":2,"dry_penalty_last_n":1024,"dry_sequence_breakers":["\n",":","\n","*"],"mirostat":0,"mirostat_tau":5.0,"mirostat_eta":0.10000000149011612,"stop":[],"max_tokens":128,"n_predict":128,"n_keep":0,"n_discard":0,"ignore_eos":false,"stream":false,"logit_bias":[],"n_probs":0,"min_keep":0,"grammar":"","grammar_lazy":false,"grammar_triggers":[],"preserved_tokens":[],"chat_format":"Content-only","reasoning_format":"deepseek","reasoning_in_content":false,"generation_prompt":"","samplers":["penalties","dry","top_n_sigma","top_k","typ_p","top_p","min_p","xtc","temperature"],"speculative.types":"none","timings_per_token":false,"post_sampling_probs":false,"backend_sampling":false,"lora":[],"prompt":"In 2 sentences, what is the Olympic Games?","has_new_line":true,"truncated":false,"stop_type":"eos","stopping_word":"","tokens_cached":40,"timings":{"cache_n":0,"prompt_n":11,"prompt_ms":1191.925,"prompt_per_token_ms":108.35681818181818,"prompt_per_second":9.228768588627641,"predicted_n":30,"predicted_ms":5145.965,"predicted_per_token_ms":171.53216666666668,"predicted_per_second":5.829810346553076}}
arduino@uno-q:~/llama.cpp$
```

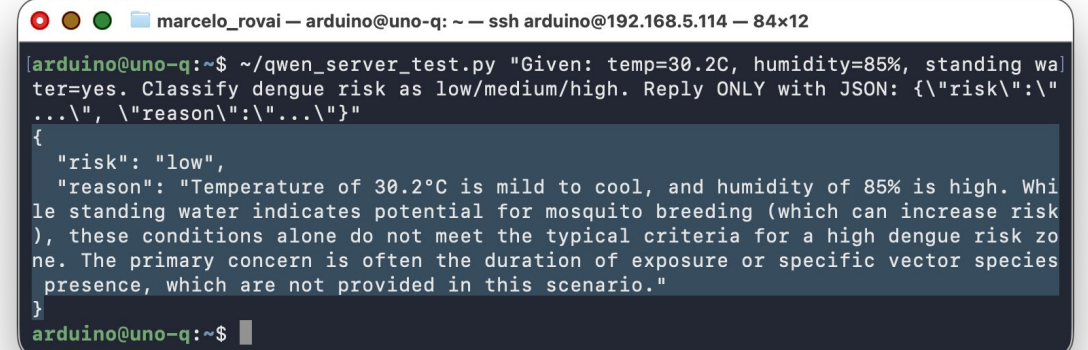
A raw completion

TERMINAL 2

The endpoint that actually matters

```
$ curl http://127.0.0.1:8081/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"model": "qwen3.5-0.8b",
  "messages": [
    {"role": "system", "content": "You are concise."},
    {"role": "user", "content": "What is TinyML?"}],
  "max_tokens": 80, "presence_penalty": 1.5}'
```

The answer is in `choices[0].message.content`. This is the OpenAI-compatible shape every Python client in the next stage will use.

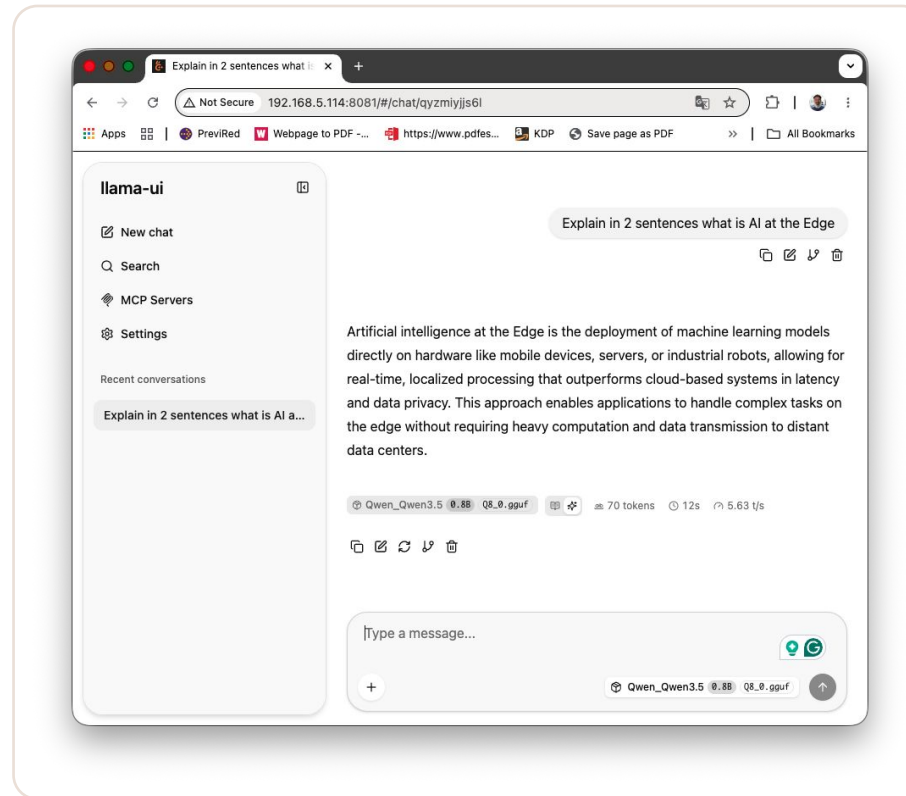


A terminal window titled 'marcelo_rovai — arduino@uno-q: ~ — ssh arduino@192.168.5.114 — 84x12'. The prompt is 'arduino@uno-q:~\$'. The command executed is `~/qwen_server_test.py "Given: temp=30.2C, humidity=85%, standing water=yes. Classify dengue risk as low/medium/high. Reply ONLY with JSON: {\"risk\": \"...\", \"reason\": \"...\"}"`. The output is a JSON object: `{\"risk\": \"low\", \"reason\": \"Temperature of 30.2°C is mild to cool, and humidity of 85% is high. While standing water indicates potential for mosquito breeding (which can increase risk), these conditions alone do not meet the typical criteria for a high dengue risk zone. The primary concern is often the duration of exposure or specific vector species presence, which are not provided in this scenario.\"\"}`. The prompt is now 'arduino@uno-q:~\$'.

The JSON response

TWO EXTRAS

The WebUI ships inside the binary



llama-server's embedded WebUI, from any device on the Wi-Fi

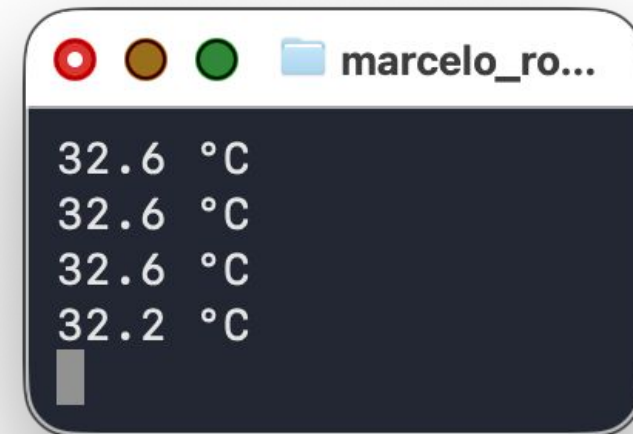
Add `--host 0.0.0.0` and open `http://<UNO_Q_IP>:8081/` — or keep it on loopback and tunnel: `ssh -L 8081:127.0.0.1:8081 arduino@<IP>`

TWO EXTRAS

Watch the temperature while it works

```
$ watch -n1 cat /sys/class/thermal/thermal_zone0/temp
```

Divide by 1000 for °C. Run it in a third session during a long generation.



Thermal zone, live

STAGE 5 OF 6

Calling it from Python

The standard openai client, then raw HTTP — so you can see what the library does underneath.

5

THE RECOMMENDED PATH

The openai client library

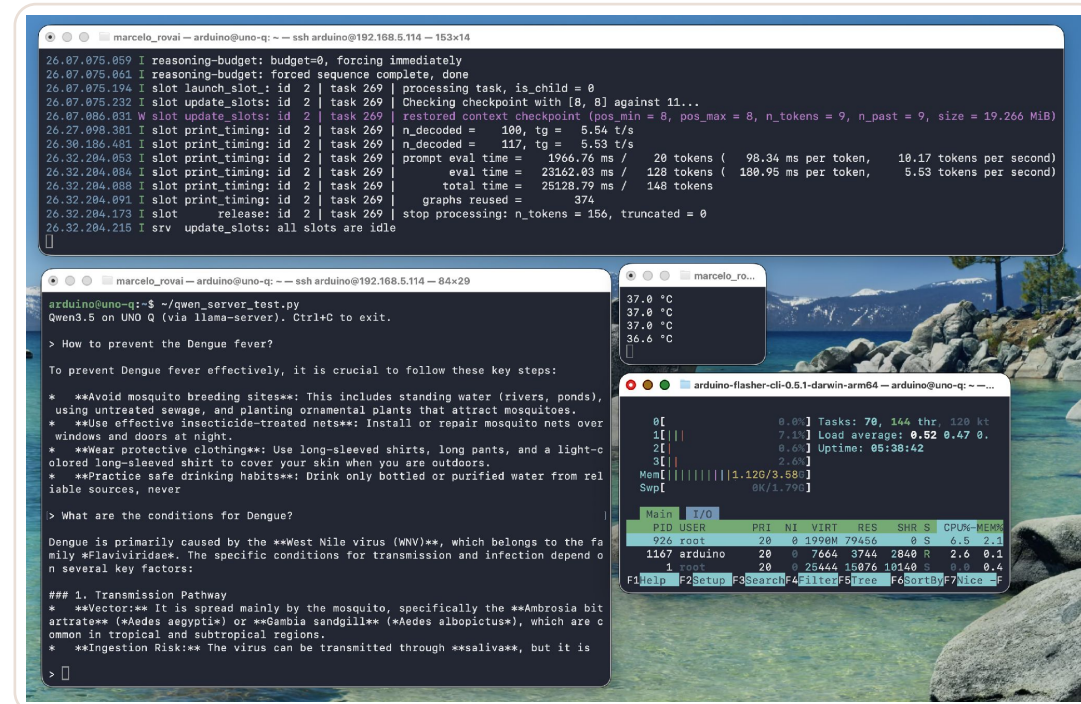
```
$ pip install --user openai
```

```
from openai import OpenAI
```

```
client = OpenAI(
    base_url="http://127.0.0.1:8081/v1",
    api_key="not-needed") # ignored, but required
```

```
response = client.chat.completions.create(
    model="qwen3.5-0.8b", messages=messages,
    max_tokens=128, stream=True)
```

llama-server ignores the API key, but the client library insists on one. Any string works.



The client, one-shot and interactive

THREE THINGS TO NOTICE

In that code

1

stream=True

Tokens print as they are generated instead of waiting for the whole response. On a 5 tok/s board this transforms perceived speed.

2

history is a plain list

Each turn appends the prompt and the reply, so the model sees the conversation next call. There is no hidden server-side state.

3

strip_think() is cosmetic

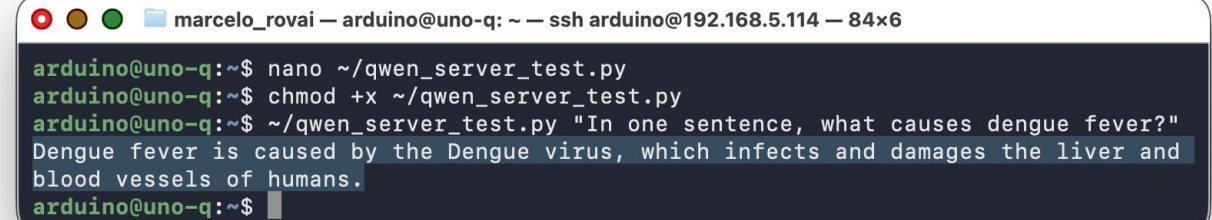
Even with reasoning off, Qwen3.5 sometimes emits an empty `<think></think>`. The content is empty — no reasoning is happening.

Multi-turn memory is just a Python list you keep appending to. That is worth saying out loud.

PATTERN 1

Ask for JSON, get JSON

```
$ ~/qwen_client.py \  
"Given: temp=30.2C, humidity=85%,  
standing water=yes.  
Classify dengue risk low/medium/high.  
Reply ONLY with JSON:  
{risk: ..., reason: ...}"
```



```
marcelo_rovai — arduino@uno-q: ~ — ssh arduino@192.168.5.114 — 84x6  
arduino@uno-q:~$ nano ~/qwen_server_test.py  
arduino@uno-q:~$ chmod +x ~/qwen_server_test.py  
arduino@uno-q:~$ ~/qwen_server_test.py "In one sentence, what causes dengue fever?"  
Dengue fever is caused by the Dengue virus, which infects and damages the liver and  
blood vessels of humans.  
arduino@uno-q:~$
```

Structured output

Valid JSON most of the time at Q8. When it is not, strip the fences and retry — the `parse_verdict()` pattern in the written chapter.

PATTERN 2

Or skip the dependency entirely

```
import requests

resp = requests.post(
    "http://127.0.0.1:8081/v1/chat/completions",
    json={"model": "qwen3.5-0.8b",
          "messages": [...],
          "max_tokens": 80,
          "presence_penalty": 1.5},
    timeout=120)
print(resp.json()["choices"][0]["message"]["content"])
```

Useful for an embedded script, a minimal container image, or just to see the wire format. It is plain JSON over HTTP — nothing magic in the library.

CHOOSING

Library or raw HTTP?

	openai library	raw requests
Code	Shorter, typed response objects	More verbose, manual JSON parsing
Streaming	Built-in iterator over chunks	Manual SSE line parsing
Moving to a cloud provider	Change <code>base_url</code> + <code>api_key</code>	Change URL and headers by hand
Dependencies	Adds the openai package	Only requests — often already installed
Error handling	Raises typed exceptions	Inspect <code>status_code</code> and <code>response.text</code>
Best for	Application code, anything long-lived	Quick scripts, debugging, tight environments

Application code can move from a local SLM to a cloud LLM and back with a one-line URL change. That portability is why we used the OpenAI-compatible surface.

STAGE 6 OF 6

Performance and limits

Measured numbers, the failure modes, and where the 0.8B model stops being enough.

6

MEASURED

What to actually expect

Cold model load	~4 s
Idle RAM, server running	~700–800 MB
Model + context	~1,122 MiB
Prompt processing	~9.9 tok/s
Generation	~4.75 tok/s
CPU during decode	4 cores @ 100%
Idle temperature	~34 °C
Inference temperature	~54 °C
Throttle threshold	70–80 °C — never reached

4.75

tokens/s generation

3.1 W

maximum power draw

THE OPTIMISATION THAT ACTUALLY PAYS

Cap max_tokens.

People optimise tokens per second when wall-clock latency is what users feel. A 30-token answer at 10 tok/s takes 3 seconds. A 100-token answer at the same rate takes 10. Design prompts to keep answers short — it is the cheapest latency win on this board.

FAILURE MODES

The four things that will go wrong

1

Model loops, or answers forever

Thinking mode is active. Check `--reasoning` off and `presence_penalty` 1.5. This accounts for most of it.

2

OOM crash during model load

Confirm swap with `free -h` and reduce `--ctx-size`. The load spike is the danger, not steady-state inference.

3

JSON parse failures

Strip fences and a leading 'json' token, then retry. Occasional stray text is normal at this size.

4

Repetition loops

`presence_penalty` = 1.5, everywhere. A documented behaviour of small Qwen3.5 variants, not a bug in your setup.

Every one of these has a specific fix. Teach the fix with the symptom and students stop guessing.

SIZING UP

When to move from 0.8B to 2B

Dimension	0.8B Q8_0	2B UD-Q4_K_XL
Model file size	~880 MB	~1.25 GB
RAM footprint (model + 1024 ctx)	~1.1 GB	~1.9–2.0 GB
Free RAM after load (of ~3 GB)	~1.7 GB	~0.8 GB
Generation speed	~4.75 tok/s	~1.8–2.5 tok/s
Quality on structured JSON	Good with a strong few-shot	Noticeably more robust
Quality on free-form chat	Adequate	Meaningfully better

Parameter count beats quantization: a 4-bit-larger model often outperforms a smaller one at higher precision, with nearly the same memory usage.

BE REALISTIC

What this setup cannot do

PRECISION

Q4 hurts here

A 0.8B model at Q4 loses more than a 7B does — less redundancy to exploit. Quality at this size is uneven: expect occasional nonsense JSON or hallucinated reasons.

HARDWARE

No NPU

The Adreno 702 GPU is not a reliable llama.cpp target. Four A53 cores do all the work. On older images, storage is tight too.

BEHAVIOUR

Thinking is unusable

Multi-minute inference and frequent loops. Always --reasoning off. Keep a human in the loop for anything safety-critical.

But the patterns port upward: the VENTUNO Q — 40 TOPS NPU, 16 GB RAM — makes billions of parameters interactive. Same llama.cpp, same HTTP API. Only the numbers change.

CARRY THESE OUT OF CHAPTER 2

Three things

1

Local generative AI, on Arduino hardware

On the same board used for Blink, a language model now answers with no internet connection.

2

Standard, transferable tools

An OpenAI-compatible API and the official client library. Nothing here is board-specific except the constraints.

3

Bounded — and that is the lesson

Small models, short outputs, batch-rate inference. Cap `max_tokens`, keep `presence_penalty` at 1.5, thinking off.

Bounded, but the boundary moves. The VENTUNO Q changes the numbers, not the patterns.

RESOURCES

Where to read further

Resource	Where
llama.cpp repository	github.com/ggml-org/llama.cpp
llama.cpp HTTP server docs	github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md
Qwen3.5-0.8B GGUF (Bartowski)	huggingface.co/bartowski/Qwen_Qwen3.5-0.8B-GGUF
Qwen3.5-0.8B GGUF (Unsloth Dynamic)	huggingface.co/unsloth/Qwen3.5-0.8B-GGUF
Unsloth Qwen3.5 inference guide	unsloth.ai/docs/models/qwen3.5
OpenAI Python library	github.com/openai/openai-python

Next — Ch. 3, Multimodal AI at the Edge: the same model given eyes. A vision projector, a vision-enabled server, and reasoning about camera input.

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI — Federal University of Itajubá, Brazil

AiEng4D — Academic Network Co-Chair

EdgeAIP — Academia-Industry Partnership Co-Chair

